Developing with GitHub Copilot

OBJECTIVES

- Place Copilot as the middleware between the editor and the model.
- Write the code that stays: comments, completion, Ctrl+I.
- Keep the agent for unit tests and POCs.
- Read a request: tokens, history, RAG, functions, tools.
- Inject an instruction, an agent, or a skill at the moment it goes.

SCHEDULE

MOMENT	SUBJECT
1	What Copilot is
2	Chat and the stable product
3	Comments, completion, Ctrl+I
4	Practice the three gestures
5	What goes into the request
6	What an agent is
7	Configurations
8	Practice: a skill, a POC

VOCABULARY

WORD	MEANING
Chat	Agentic coding. The agent writes the code
Ask and Ctrl+I	Are not called chat

DEVELOPING WITH GITHUB COPILOT

Introduction

DEFINITION — CHAT

The agent that writes the code. Ask and Ctrl+I are not called chat.

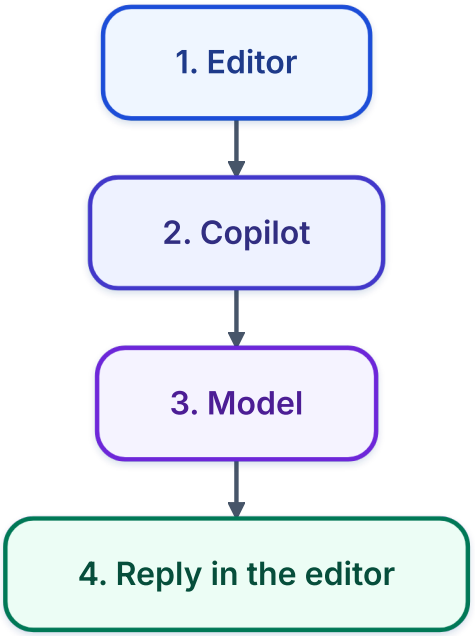
- Presentation: About GitHub Copilot
- Customize: Response customization

Module 1 — What Copilot is

Copilot is not the model.

DEFINITION — MIDDLEWARE

Layer between the editor and the model. It builds the request. It sends it. It puts the reply back.



- The editor gives the cursor and the open files.
- The model answers this text alone.
- The model does not see the project.
- The model does not see the closed files.
- The model can change. This layer stays.
- A copied excerpt enters the request.

GLOSSARY

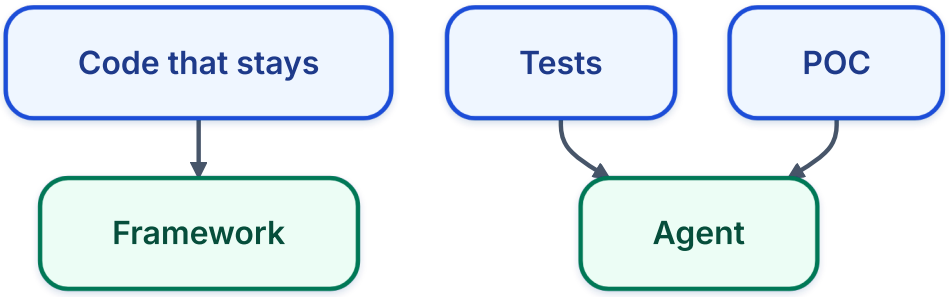
TERM	MEANING
Copilot	The middleware. Not the model
Model	Reads the request. Produces the reply
Request	The only text the model receives
Chat	The agent that writes the code

Module 2 — Chat and the stable product

DEFINITION — CHAT

The agent that writes the code. Ask and Ctrl+I are not called chat.

- The code that stays is not written in chat.
- Quality and stability matter.
- What is not written is decided in silence.
- Codebase knowledge is lost.
- The framework already saves time.
- Comments. Then completion. Then Ctrl+I.
- Quality code is faster this way. It is also cheaper.



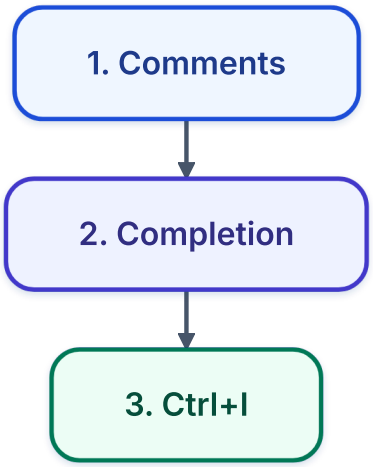
- The agent writes unit tests.
- The agent makes a POC.
- The agent does not write the code that stays.

MEASURES

SOURCE	FINDING
Ford, TechSparks 2025	85% coverage. More than 2,000 tests for 40 lines. Build from 10 to 25 minutes. Redundant code in production. Human review comes back
CodeRabbit, 470 PR	10.83 problems per PR on the AI side. 6.45 on the human side
Veracode, 2025	OWASP Top 10 flaw in 45% of tasks
DORA, 2024	+25% AI use: about +7.2% instability
Replit, July 2025	Explicit freeze. Production database erased
PocketOS, April 2026	Production volume deleted. Nine seconds
GitClear, 2025	Moved code: 25% in 2021, less than 10% in 2024
METR, July 2025	19% more time. Forecast: 24% less. Feeling of going faster
Godot, 2026	Agent contributions refused. Completion kept
GitHub, 1 June 2026	Tokens billed. One agent session costs more than a short question

Module 3 — Comments, completion, Ctrl+I

The code that stays is written with three gestures. Always in this order.



- The algorithm is in comments. In the file. Before the code.
- Completion implements what is written.

DEFINITION — COMPLETION

A suggestion at the cursor. It reads the nearby comment. It does not choose the decomposition.

- Each keystroke is a new request.
- Ctrl+I adjusts code already written. Ctrl+I is not chat.
- This framework saves time.

THE KEYS

KEY	EFFECT
Tab	Accept the suggestion.
Escape	Dismiss the suggestion.
Ctrl+→	The next word. Or the next line.
Cmd+→	The same gesture, on Mac.

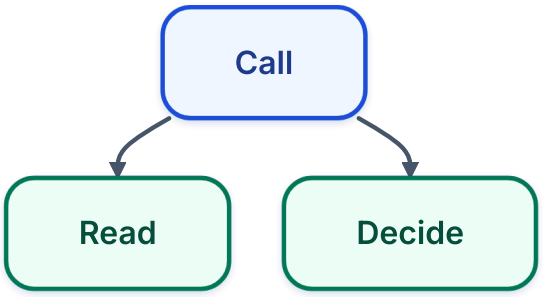
Inline suggestions · Keyboard shortcuts

SNOOZE

- Copilot menu in the Status Bar.
- Each press adds 5 minutes.
- Completion stays quiet during that time.
- Command Palette: « Snooze Inline Suggestions ».
- Resume: « Cancel Snooze Inline Suggestions ».
- No default shortcut.

DECOMPOSITION

- One function. One responsibility.
- Two functions in a chain only call.
- They do not redo the work.



- The caller does not read again.
- It does not decide again.

Module 4 — What goes into the request

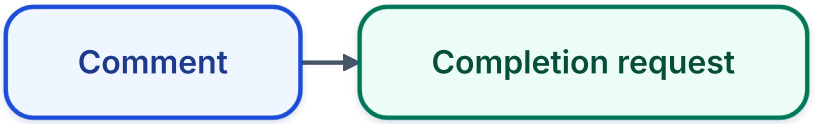
DEFINITION — TOKEN

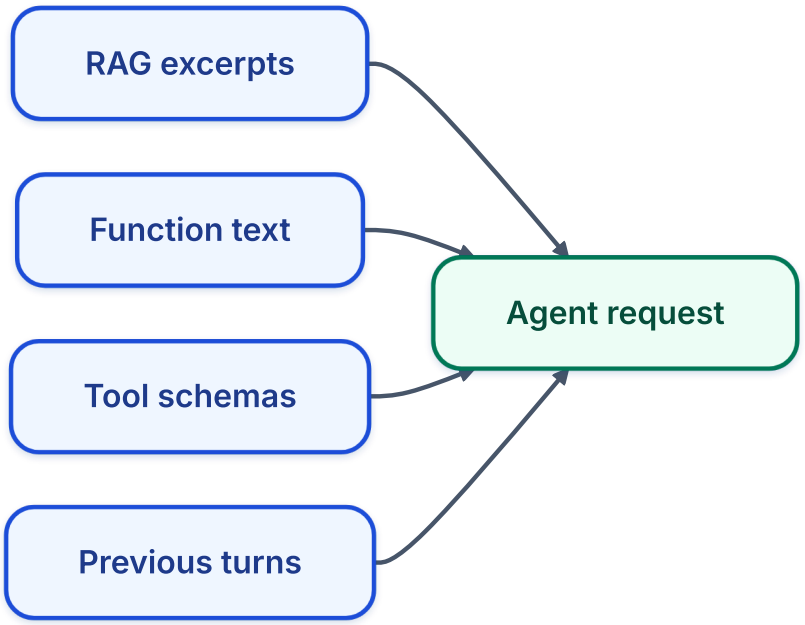
Unit the model reads and produces. Neither a character. Nor a word.

EXAMPLE

Add it if the identifier is absent.

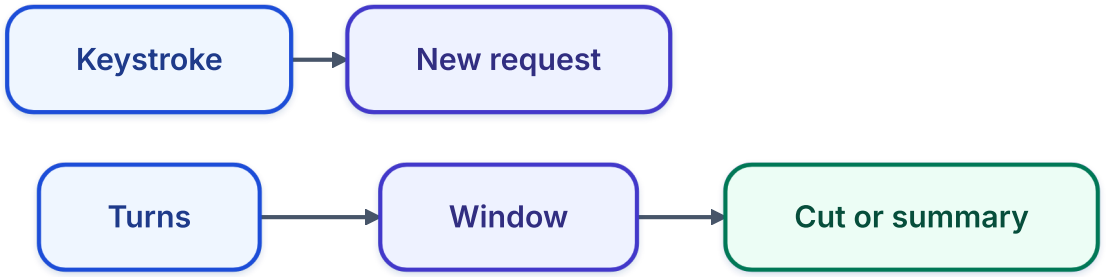
- Not one token per letter.
- Not necessarily one token per word.
- The split depends on the model.
- The window is counted in tokens.
- Each item placed in the request uses tokens.





HISTORY

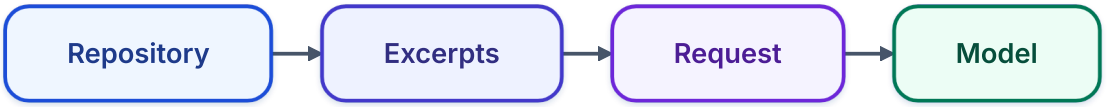
History: the turns sent back in a request. Neither the editor. Nor the commits.



- Completion has no history. Each keystroke is a new request.
- The comment next to the cursor goes into this request.
- The agent resends previous turns, as long as they fit in the window.
- Beyond that, the oldest ones are cut or summarized.

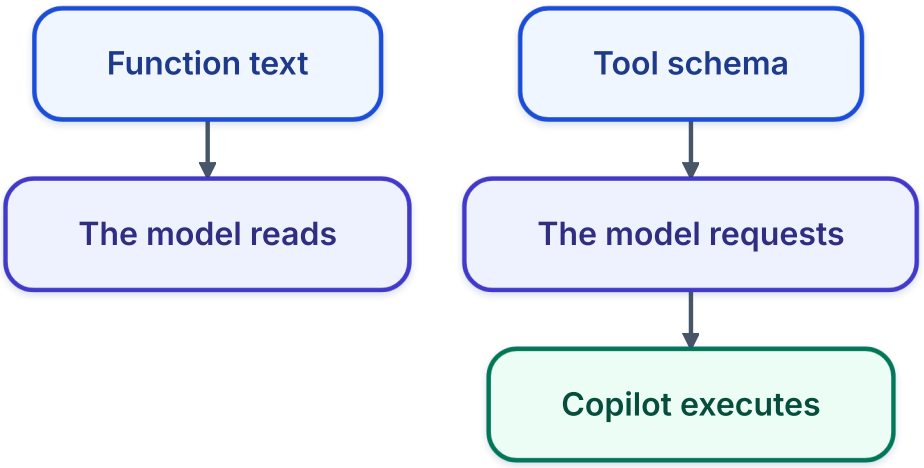
RAG

> **DEFINITION — RAG**
> Search for repository excerpts. These excerpts are pasted into the request. The model has no repository memory.



- The repository is searched. Excerpts are pasted. The model reads these excerpts.
- A closed file does not go in just because it is in the folder.

TWO LISTS

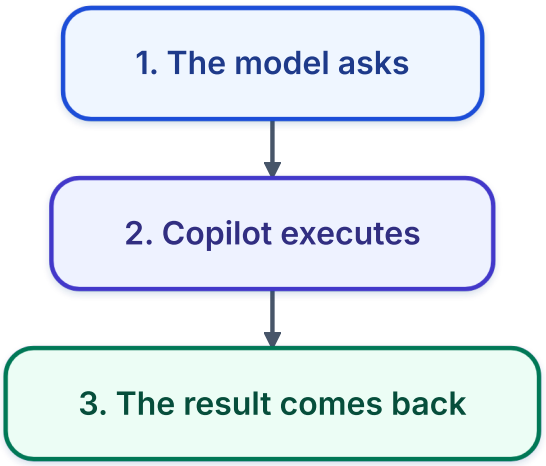


- A function written in the file is text.
- The tool that runs a test is a schema.
- The model requests the schema. Copilot executes it.

Module 5 — What an agent is

An agent is the same middleware, with a loop.

- Copilot builds the request.
- It sends it.
- It puts it back.
- Then the loop starts again.



- The model asks for a tool.
- Copilot executes it.
- The result comes back into the next request.

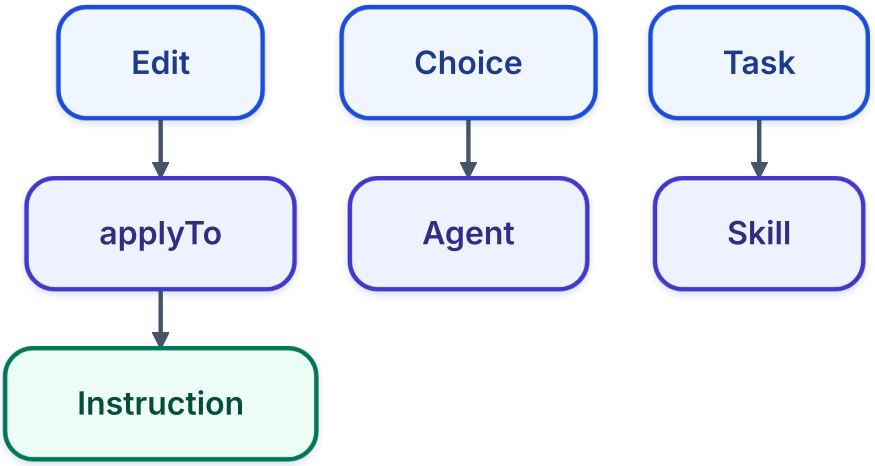
> **DEFINITION — TOOL**
>
> Read. Search. Edit. Run a test.

TOOL	EFFECT
Read	The text comes back into the next request
Search	The excerpts come back
Edit	The change is applied
Run a test	The result comes back

- RAG puts excerpts back on each turn.
- The agent does not have the project in memory.
- Full window: old turns are cut or summarized.
- The agent writes the unit tests. The agent makes a POC.
- The code that stays: comments, completion, Ctrl+I.

Module 6 — Configurations

A configuration is a prompt. It enters the request at a precise moment.



- Instructions do not go on every request.
- They go when an edit is sent.
- Only if the touched file matches `applyTo`.

> **DEFINITION — APPLYTO**
> Path pattern. File `*.instructions.md`. The instruction goes if the file matches.

- A configured agent goes at the moment of the choice.
- It carries the role and the tool list.
- It does not go before that choice.
- A skill goes only when the task matches its description.
- Instruction tokens are paid on that edit.
- The skill is paid only if it is loaded.

CONFIGURATION	FILE
Instruction	`.github/instructions/` `*.instructions.md`
Agent	`.github/agents/` `*.agent.md`
Skill	`.github/skills/<name>/` `SKILL.md`

THE HEADERS

- Each type has its header list.
- The page gives the complete list.

TYPE	HEADERS	LIST
Instruction	`name` `description` `applyTo`	Header list
Agent	`description` `name` `tools`	Header list
Skill	`name` `description`	Header list